CSC3100: Trees

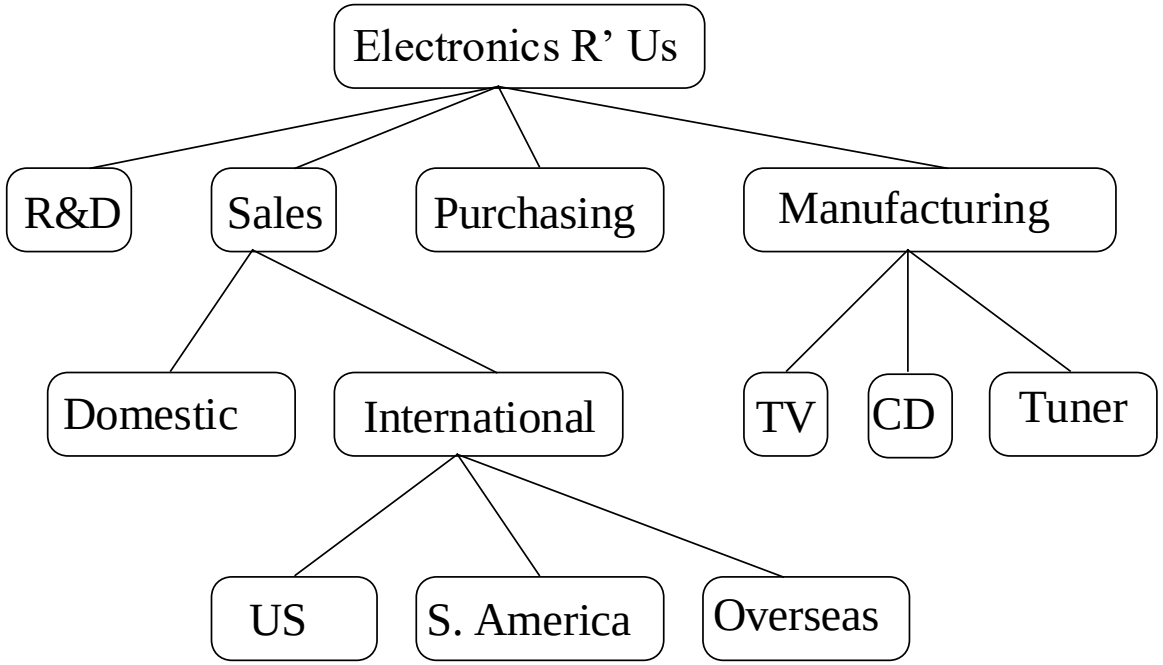
Tianshu Yu

2024 Fall

Tree

What is a Tree?

- Organization of a company



Definitions (Recursive)

- A tree is a collection of nodes.
- The collection can be empty .
- Otherwise, a tree consists of a distinguished node *r*, called root, and zero or more nonempty (sub) trees T_1, T_2, … T_k, each of whose roots are connected by a direct edge from *r*.
- A tree with *N* nodes has one root, and *N*−*1* edges

Definitions

- Parent

  Node A is the parent of node B if B is the root of the left or right sub-tree of A
- Left (Right) Child

  Node B is the left (right) child of node A if A is the parent of B.
- Sibling

  Node B and node C are siblings if they have the same parent.
- Leaf

  A node is called a leaf if it has no children

Definitions

- A path from node n_1 to n_k
  - A sequence of nodes $n_1, n_2, \ldots, n_k$ such that n_i is the parent of n_{i+1} for $1 \leq i < k$.
- Length of a path
  - The length of this path is the number of edges on the path, namely k-1.
  - Notice that in a tree, there is exactly one path from the root to each node.

Definitions

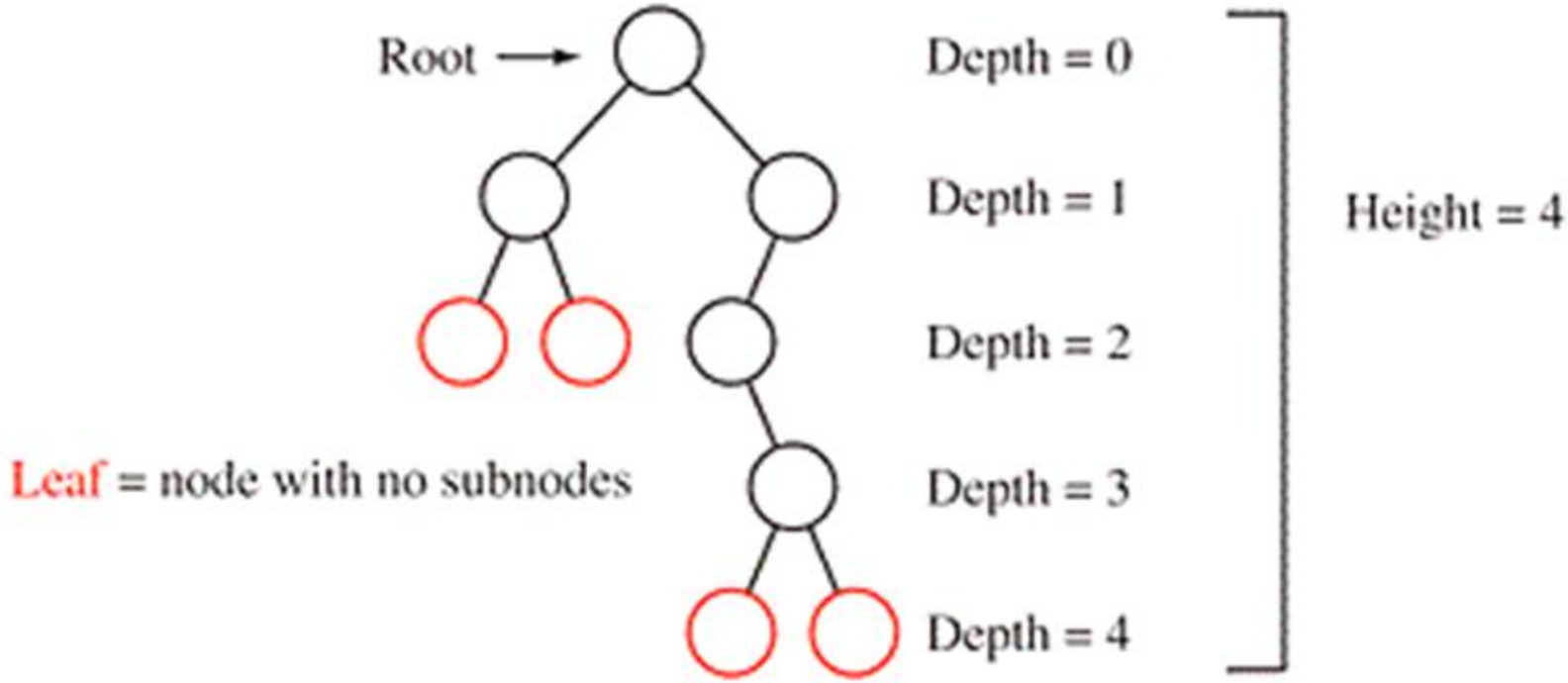
- Depth of a node n_i
  - is the length of the unique path from the root to n_i.
  - the root is at depth 0.
- Height of a node n_i
  - The height of n_i is the length of the longest path from n_i to a leaf.
  - All leaves are at height 0.

Note 1: The height of a tree is equal to the height of the root.

Note 2: The depth of a tree = the depth of the deepest leaf.

Note 3: The maximum height is equal to the maximum depth

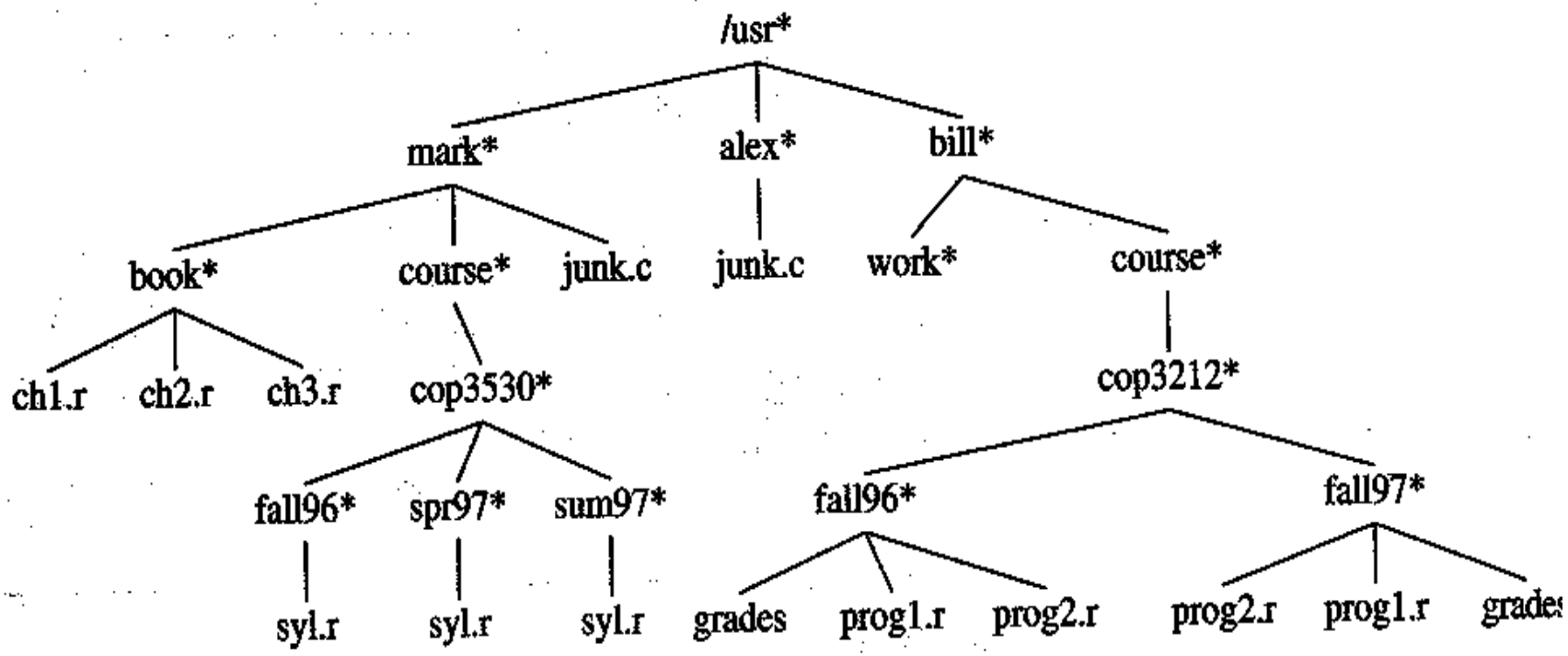
Example


Root
Depth = 0
Depth = 1
Depth = 2
Depth = 3
Depth = 4
Height = 4
Leaf = node with no subnodes

Remark

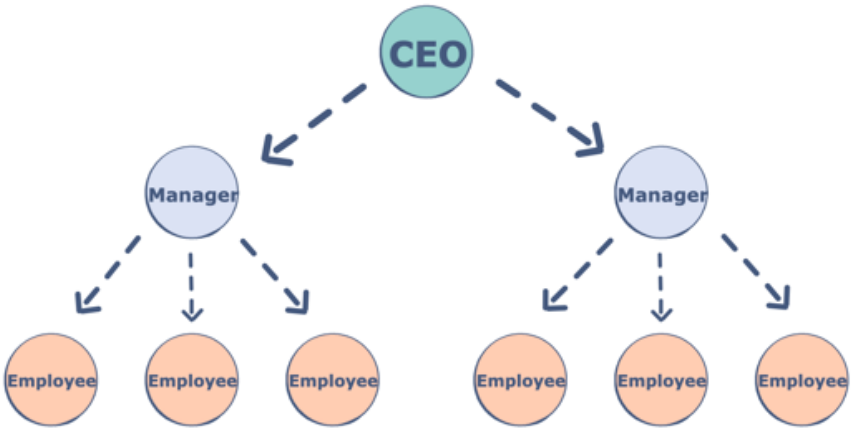
- Trees can have only one parent
- Trees have NO cycles

Applications – Unix file system



More applications

- HR system



- Java data types

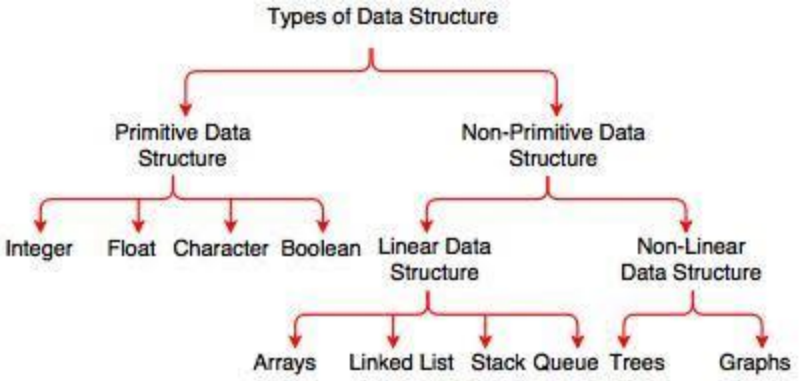


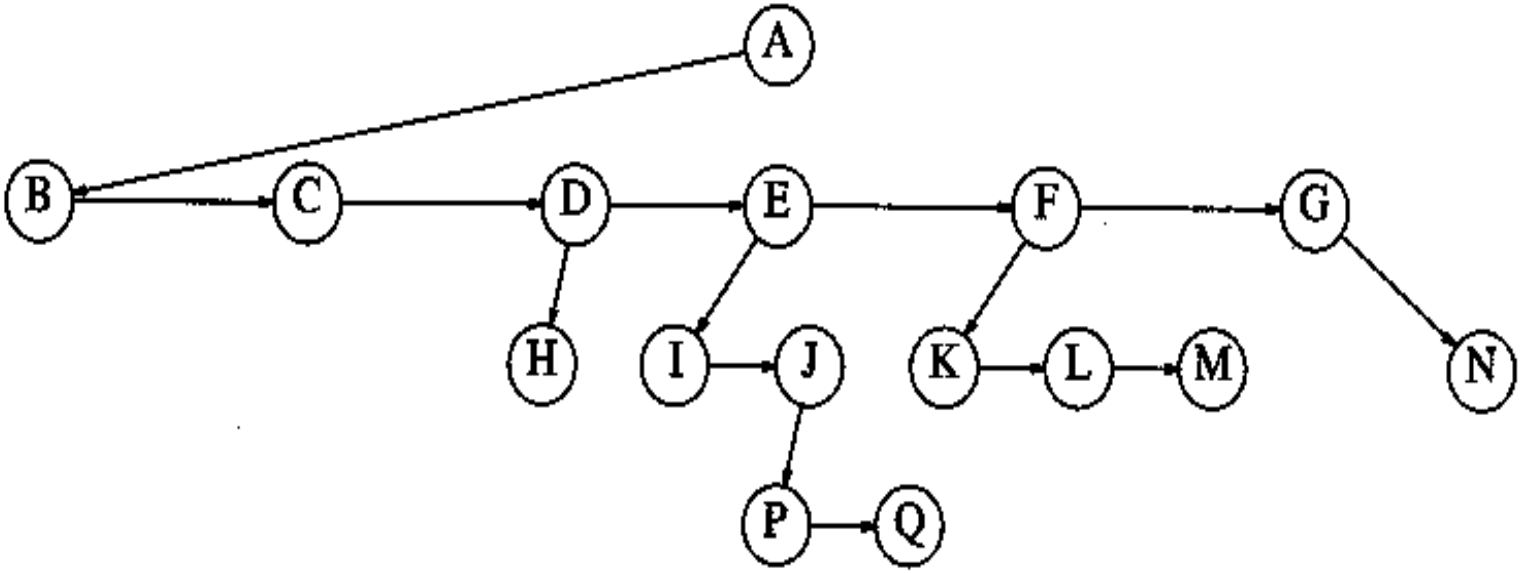
Fig. Types of Data Structure

A generic implementation

```
class TreeNode {
        Object element;
        TreeNode firstChild;
        TreeNode nextSibling;
}
```

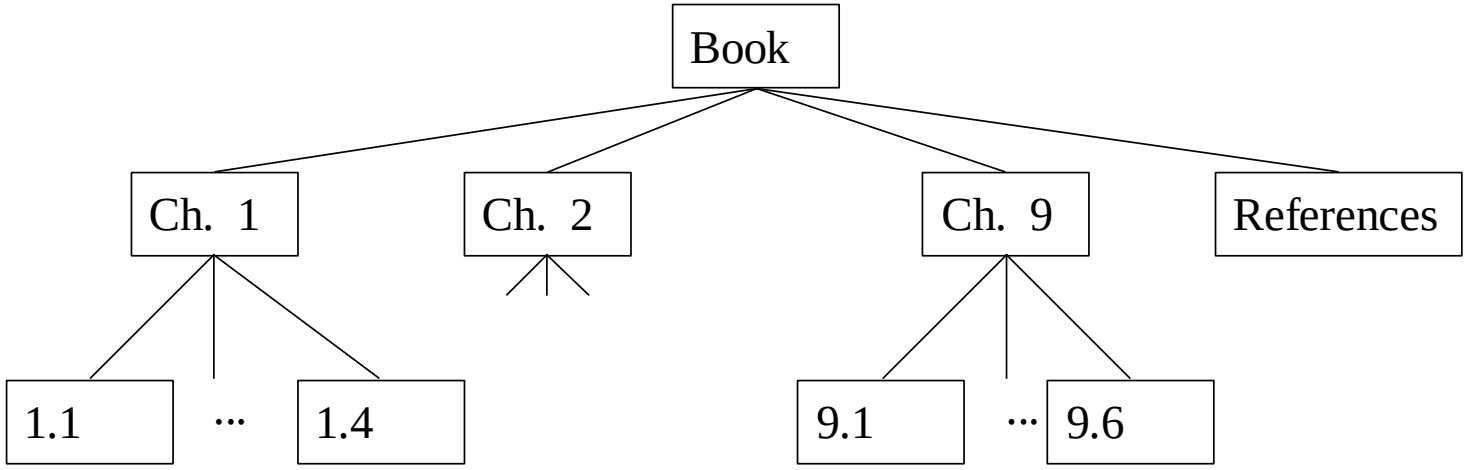
A generic implementation

- firstChild pointer: arrow that points downward
- nextSibling pointer: arrow that goes left to right



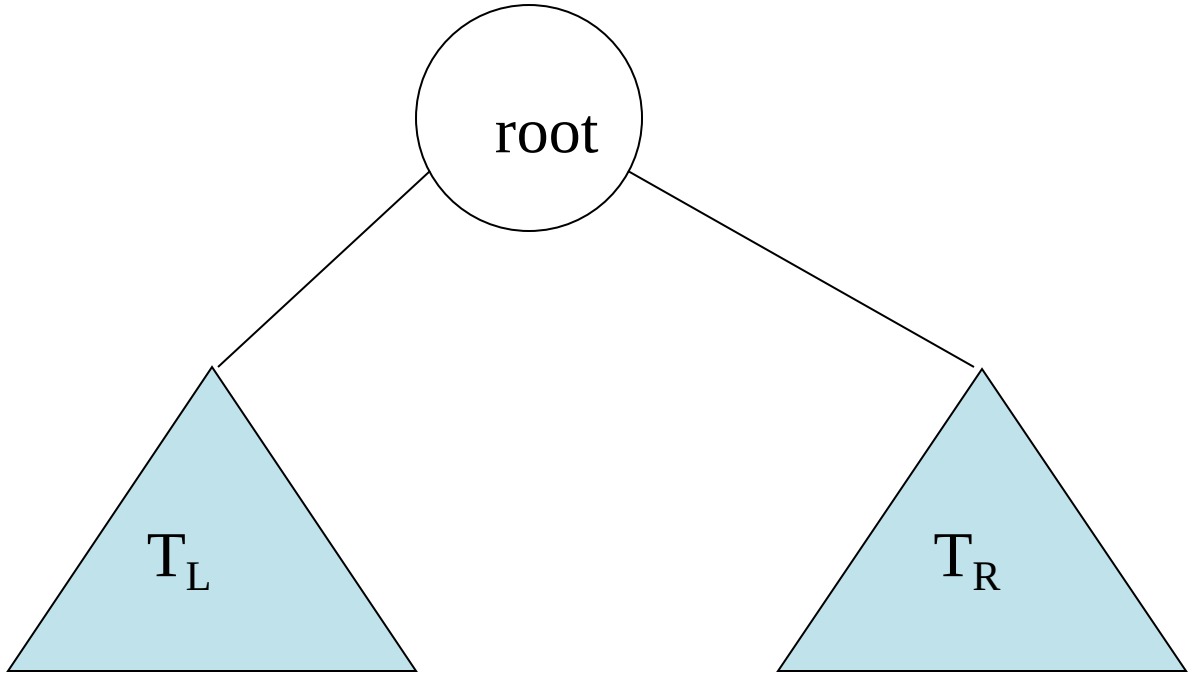
Ordered tree

- There is a linear ordering defined for the children of each node; that is, we can identify children of a node as the first, second, third, and so on.
- Example: The structure of a book is an ordered tree



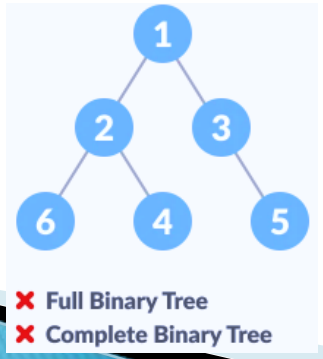
Binary tree

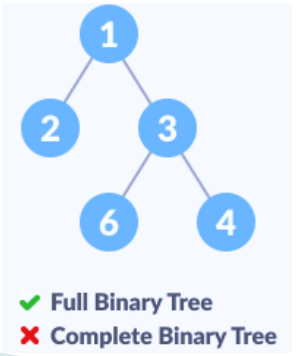
- A binary tree is a tree
  - in which no node can have no more than two children (subtrees): T_L and T_R, both of which could possibly be empty.

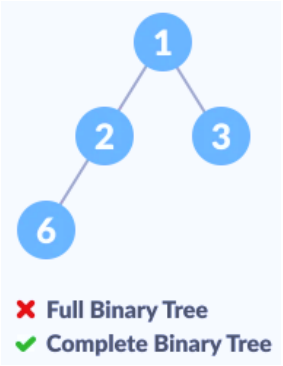


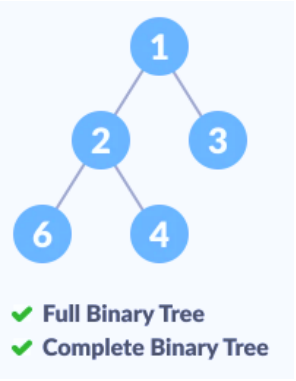
Binary tree

- Full binary tree
  - A binary tree where all the nodes have either two or no children
- Complete binary tree
  - A binary tree where all the levels are completely filled except possibly the lowest one, which is filled from the left
- Perfect binary tree
  - A tree with depth k and 2^k-1 nodes



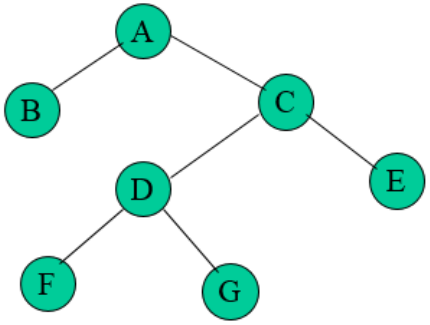




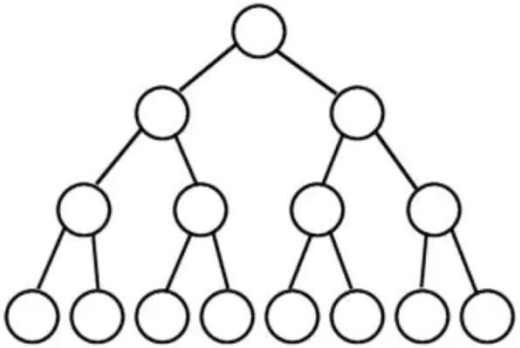


Binary tree

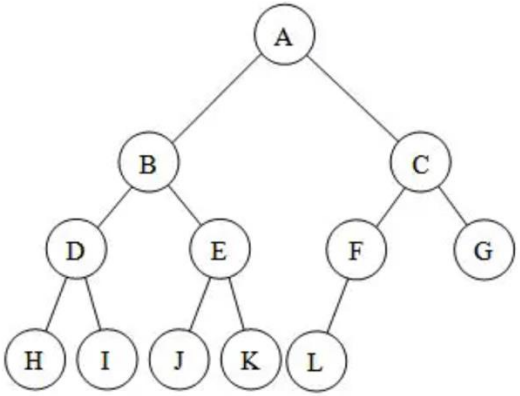
Full binary tree



Perfect binary tree



Complete binary tree



Binary tree ADT

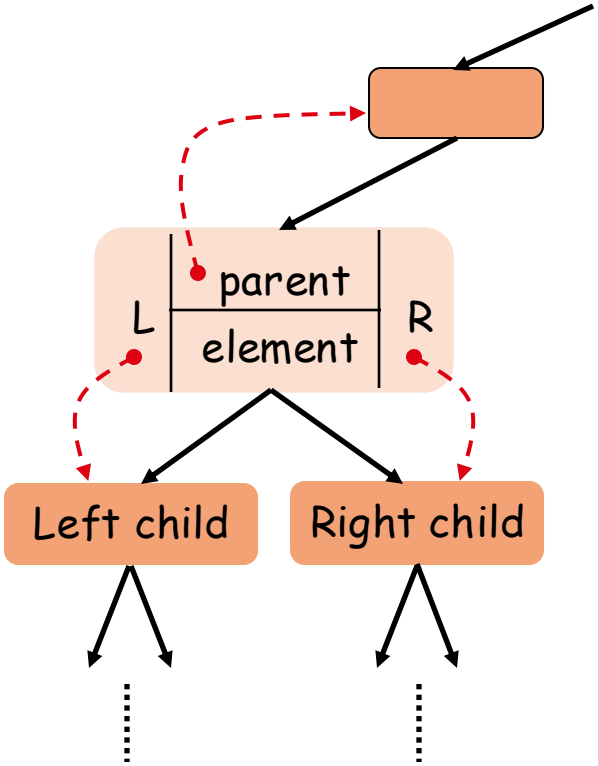
- Operations:
  - Create(bintree): creates an empty binary tree
  - Boolean IsEmpty(bintree): if bintree is empty return TRUE else FALSE
  - MakeBT(bintree1,element,bintree2): return a binary tree whose left subtree is bintree1 and right subtree is bintree2, and whose root node contains the data element
  - Lchild(bintree): if bintree is empty return error else return the left subtree of bintree
  - Rchild(bintree): if bintree is empty return error else return the right subtree of bintree
  - Data(bintree): if bintree is empty return error else return the element data stored in the root node of bintree

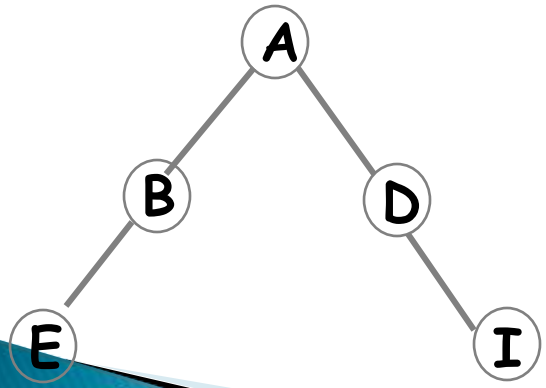
Binary tree design

- Two solutions
  - Using pointers
    - More intuitive solution
    - We will see the pseudo-codes
  - Using array
    - Need more complicated design, and cannot efficiently handle all operations (thus will omit its implementations for each operation)
    - Will be used for heap, a special type of complete binary tree

Binary tree design (i)

- A pointer representation
  - For each node node, we maintain
    - node.parent: store the address its parent,
    - node.leftchild: store the address of its left child,
    - node.rightchild: store the address of its right child
    - node.element: store the values



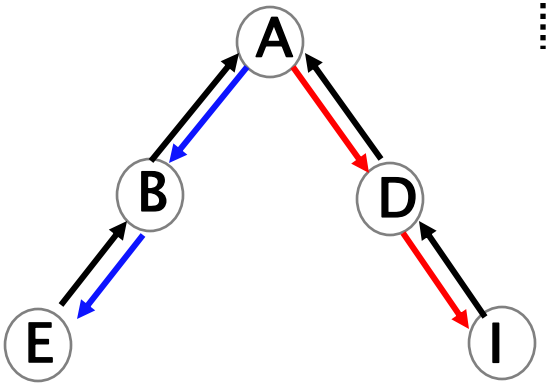


parent

leftchild

rightchild

(Omitted links points to NULL)



Binary tree: pointer implementation

- Create(bintree)

Algorithm: ***create(bintree)***

```
1  bintree = NULL
```

- isEmpty(bintree)

Algorithm: ***isEmpty(bintree)***

```
1  return bintree == NULL
```

- MakeBT(bintree1, element, bintree2)

Algorithm: ***MakeBT(bintree1, element, bintree2)***

```
1  rootNode <- allocate new memory
2  rootNode.element = element
3  rootNode.parent = NULL
4  rootNode.leftchild = bintree1
5  rootNode.rightchild = bintree2
6  if bintree1 != NULL
7    bintree1.parent = rootNode
8  if bintree 2 != NULL
9    bintree2.parent = rootNode
10 return  rootNode
```

Binary tree: pointer implementation

- Lchild(bintree)

Algorithm: ***Lchild(bintree)***

```
1 if bintree == NULL
2   error “empty tree”
3 return bintree.leftchild
```

- Rchild(bintree)

Algorithm: ***Rchild(bintree)***

```
1 if bintree == NULL
2   error “empty tree”
3 return bintree.rightchild
```

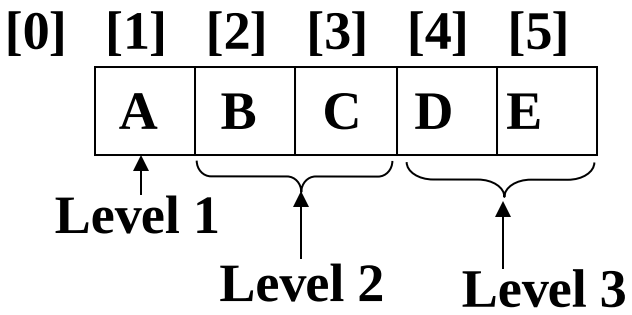
- Data(bintree)

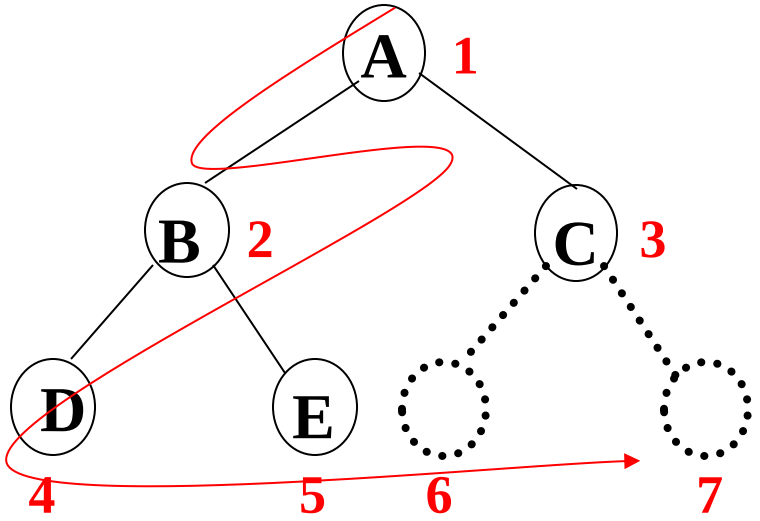
Algorithm: ***Data(bintree)***

```
1 if bintree == NULL
2   error “empty tree”
3 return bintree.element
```

Binary tree design (ii)

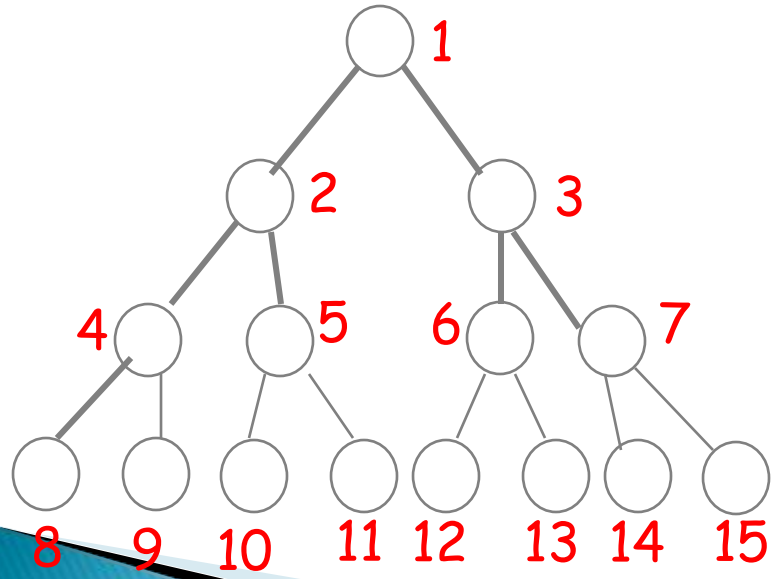
- An array representation
  - Given a complete binary tree with n nodes, for any i-th node, $1 \leq i \leq n$,
    - parent(i) is $\lfloor i/2 \rfloor$
    - leftChild(i) is at $2i$ if $2i \leq n$; otherwise, i has no left child
    - rightChild(i) is at $2i+1$ if $2i+1 \leq n$; otherwise, i has no right child

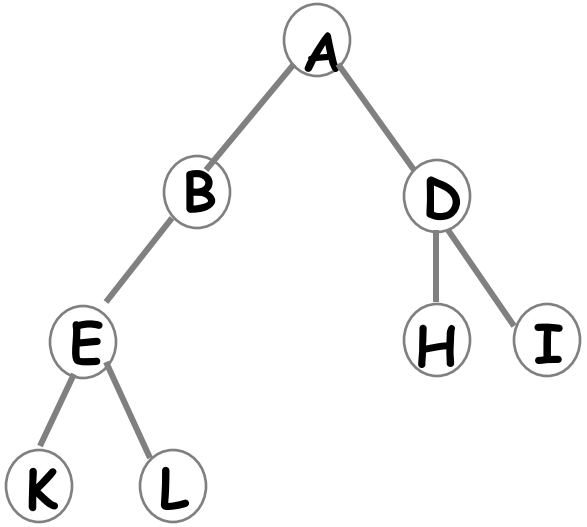




Binary tree design (ii)

- An array representation
  - Generalize to all binary trees
  - Efficient for complete binary trees
  - But inefficient for skewed binary trees
  - Inefficient to implement the ADT



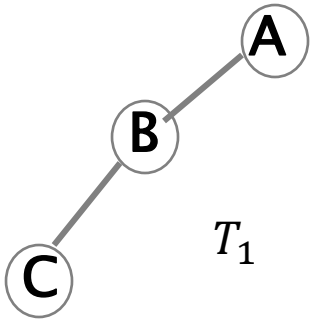


1	A	Level 1
2	B	Level 2
3	D	
4	E	Level 3
5		
6	H	
7	I	
8	K	Level 4
9	L	
10		
11		
12		
13		
14		
15		

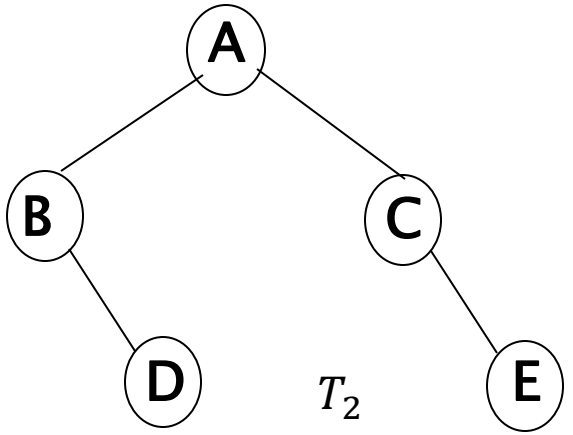
full binary tree

Practice

- What are the array representation of the following binary trees?
  - Show the content in the array
  - Hint: first obtain the ID for each node



T_1

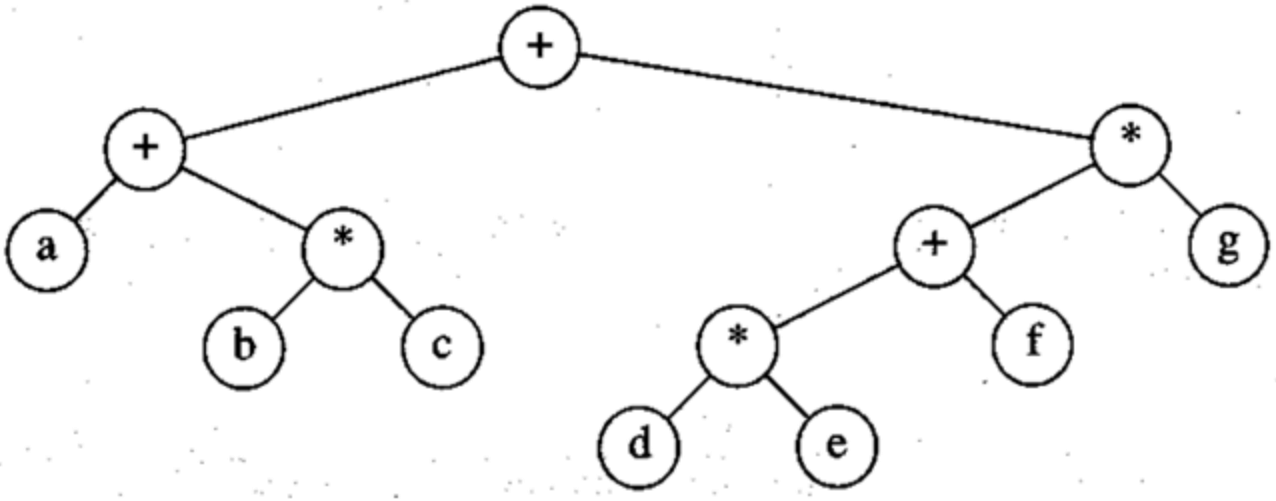


T_2

	[1]	[2]	[3]	[4]	[5]	[6]	[7]
arr							

Parsing tree

- (a+b*c) +((d*e+f)*g)



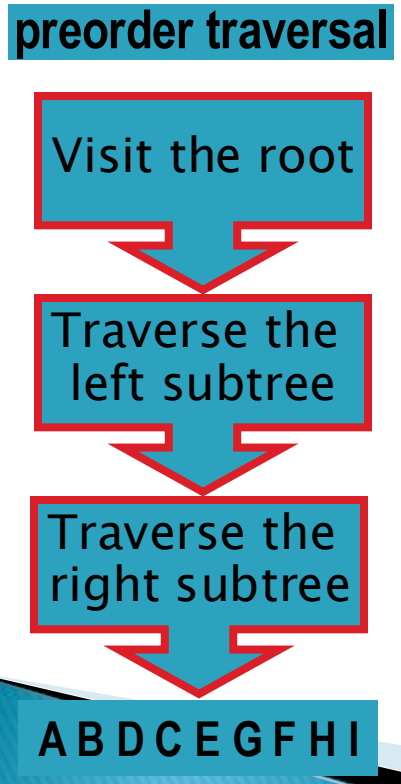
1. The leaves of an parsing tree are operands;
2. The other nodes contain operators.

Traversing Strategy

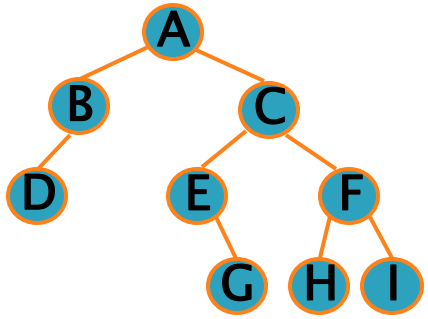
- Preorder (depth-first)
  - Visit the node
  - Traverse the left subtree in preorder
  - Traverse the right subtree in preorder
- Inorder
  - Traverse the left subtree in inorder
  - Visit the node
  - Traverse the right subtree in inorder
- Postorder
  - Traverse the left subtree in postorder
  - Traverse the right subtree in postorder
  - Visit the node

Traversing Binary Tree

When the binary tree is empty, it is "traversed" by doing nothing, otherwise:



Example:

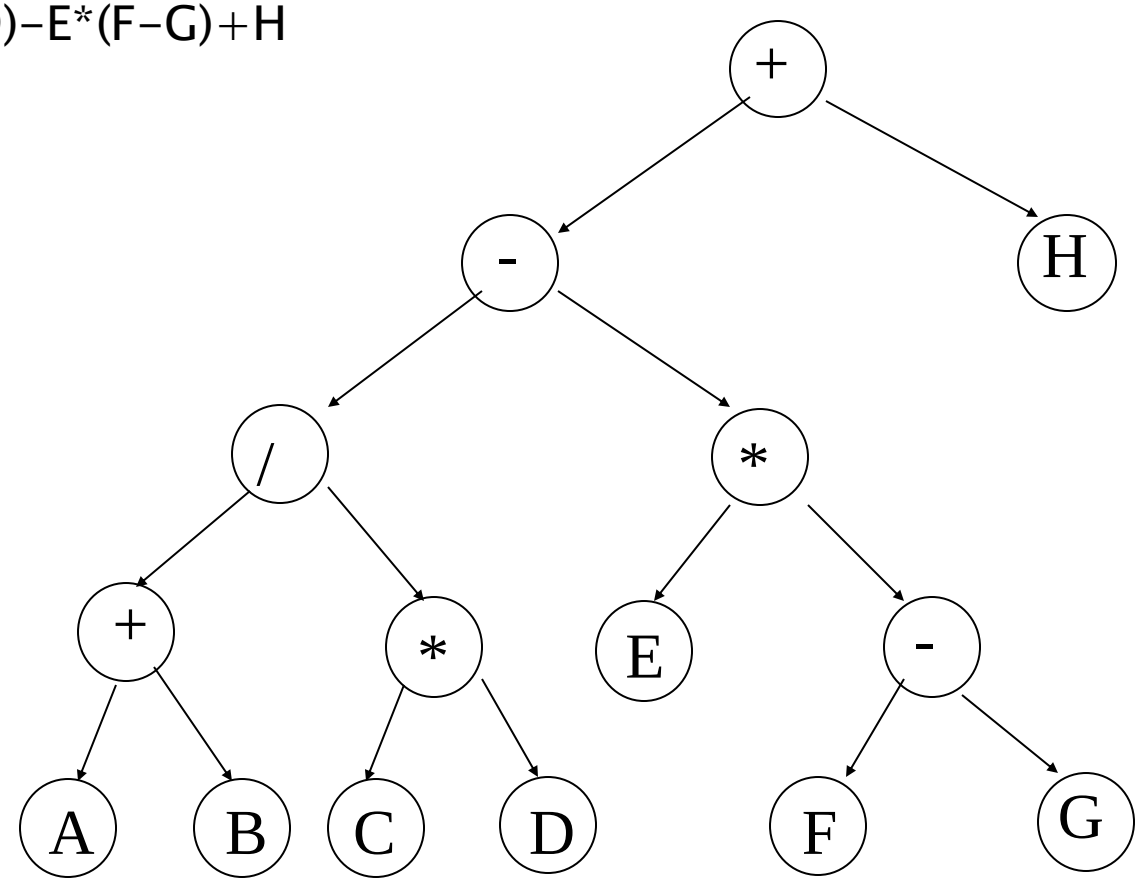


Result:

= A (A's left) (A's right)
= A B (B's left) (B's right = NULL) (A's right)
= A B (B's left) (A's right)
= A B D (D's left=NULL) (D's right = NULL) (A's right)
= A B D (A's right)
= A B D C (C's left) (C's right)
= A B D C E (E's left=NULL) (E's right) (C's right)
= A B D C E (E's right) (C's right)
= A B D C E G (G's left=NULL) (G's right = NULL) (C's right)
= A B D C E G (C's right)
= A B D C E G F (F's left) (F's right)
= A B D C E G F H (H's left=NULL) (H's right =NULL) (F's right)
= A B D C E G F H I (I's left=NULL) (I's right =NULL)
= A B D C E G F H I

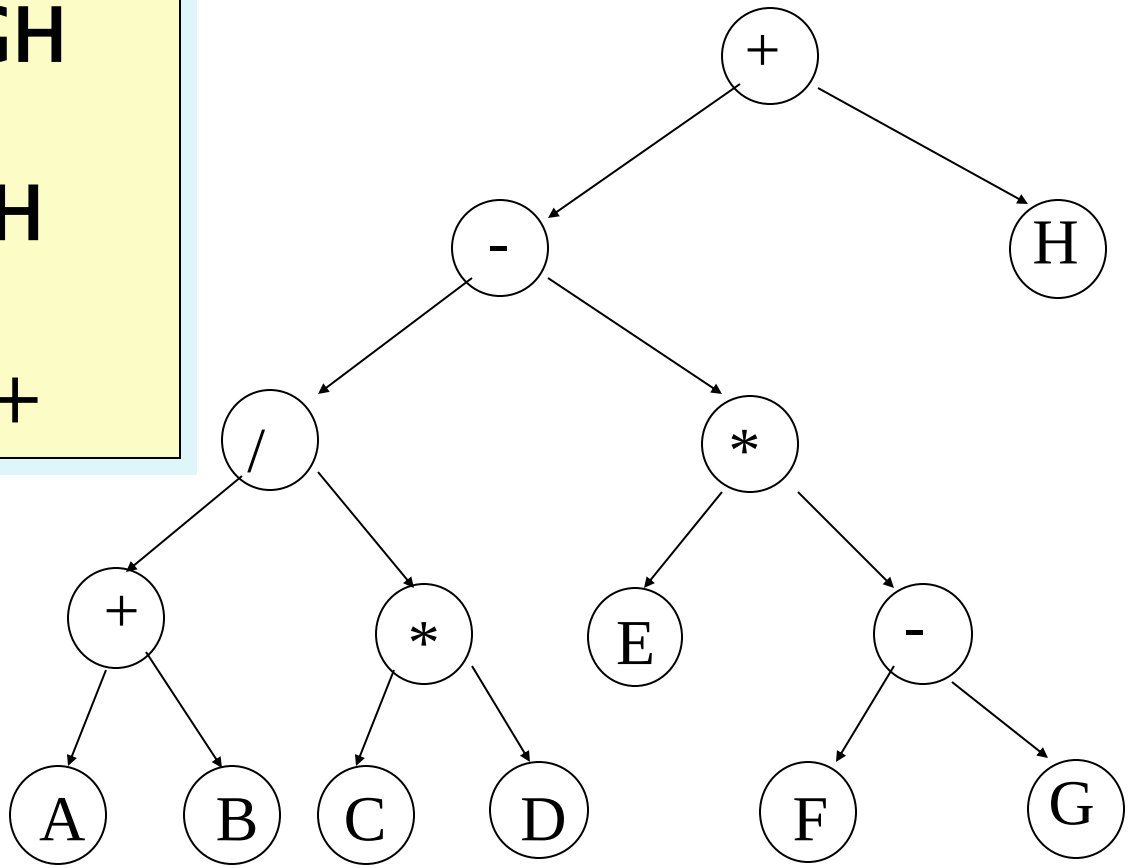
Example

(A+B)/(C*D)-E*(F-G)+H



Example

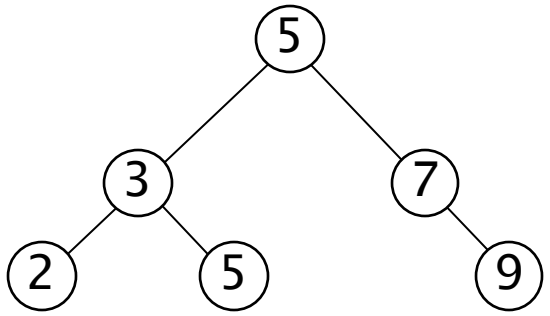
Preorder:
+-/+AB*CD*E-FGH
Inorder :
A+B/C*D-E*F-G+H
Postorder:
AB+CD*/EFG-*-H+



Implementation

```
INORDER-TREE-WALK(x)
1.    if x ≠ NIL
2.       then INORDER-TREE-WALK ( left [x])
3.            print key [x]
4.            INORDER-TREE-WALK ( right [x])
```

- *E.g.:*



Output: 2 3 5 5 7 9

- Running time:
  - $\Theta(n)$, where n is the size of the tree rooted at x

Traversing Binary Tree

Reconstruction of Binary Tree from its preorder and Inorder sequences

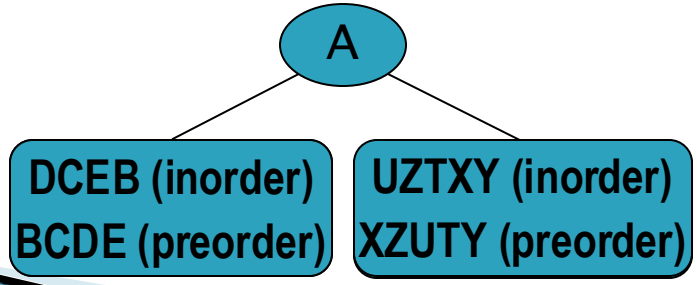
Example: Given the following sequences, find the corresponding binary tree:

preorder : ABCDEXZUTY
inorder : DCEBAUZTXY

Looking at the whole tree:

- "preorder : **A**BCDEXZUTY"
  ==> A is the root.
- Then, "inorder : DCEB**A**UZTXY"

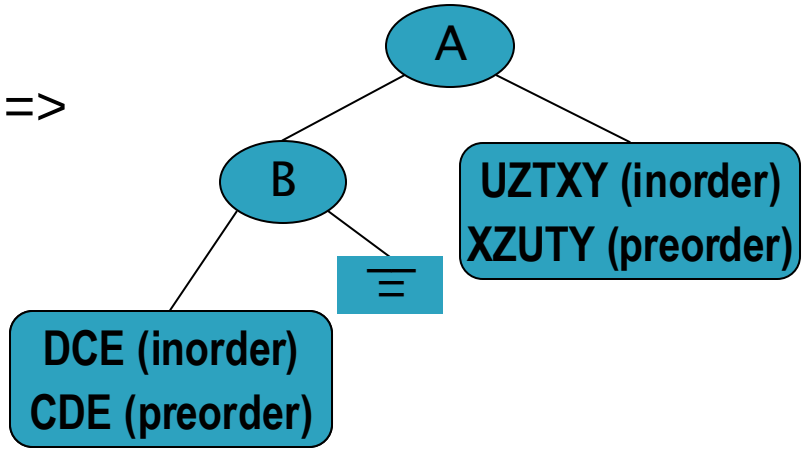
==>



Looking at the left subtree of A:

- "preorder : BCDE"
  ==> B is the root
- Then, "inorder: DCE**B**"

=>



Traversing Binary Tree

Reconstruction of Binary Tree from its preorder and Inorder sequences

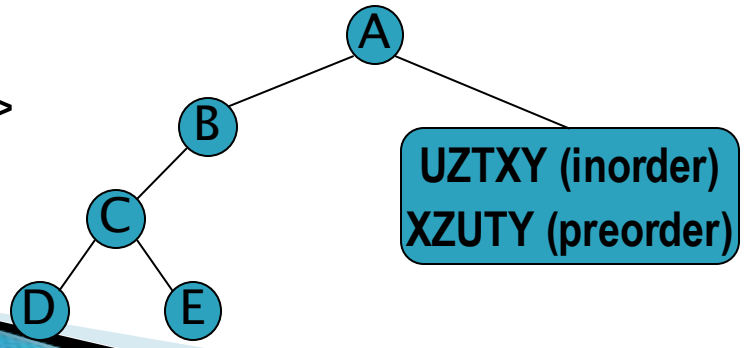
Example: Given the following sequences, find the corresponding binary tree:

preorder : ABCDEXZUTY
inorder : DCEBAUZTXY

Looking at the left subtree of B:

- "preorder : CDE"
  ==> C is the root
- Then, "inorder: D**C**E"

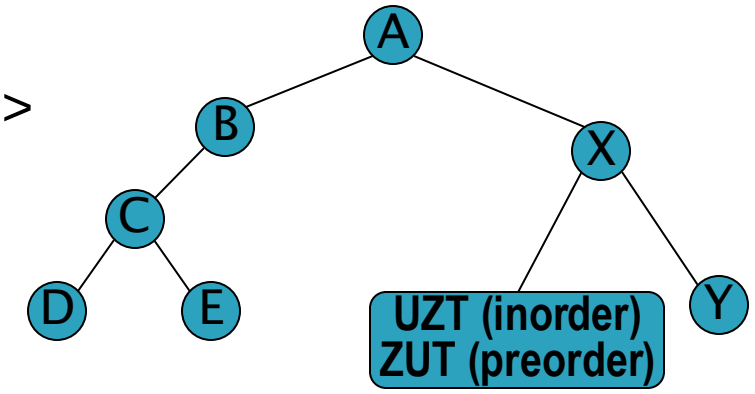
=>



Looking at the right subtree of A:

- "preorder : XZUTY"
  ==> X is the root
- Then, "inorder: UZT**X**Y"

=>



Traversing Binary Tree

Reconstruction of Binary Tree from its preorder and Inorder sequences

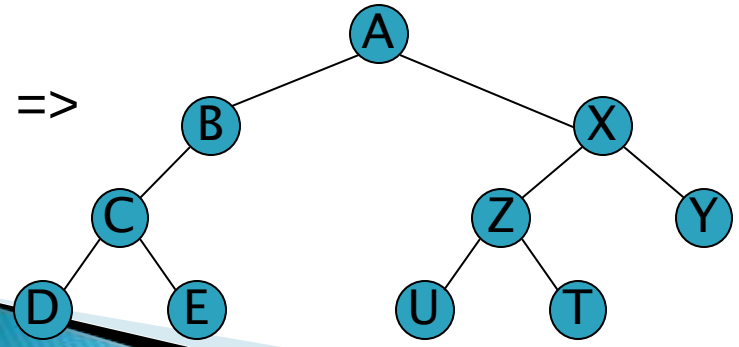
Example: Given the following sequences, find the corresponding binary tree:

preorder : ABCDEXZUTY
inorder : DCEBAUZTXY

Looking at the left subtree of X:

- "preorder : ZUT"
  ==> Z is the root
- Then, "inorder: U**Z**T"

=>

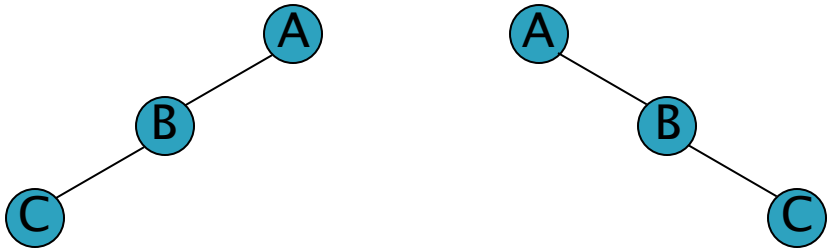


Traversing Binary Tree

But: A binary tree may not be uniquely defined by its preorder and postorder sequences.

Example: **Preorder sequence:** **ABC**
Postorder sequence: **CBA**

We can construct 2 different binary trees:



Java implementation codes

```
1.  class BTreeBuilder {
2.     private static int find(char[] array, int v){
3.        for (int i = 0;i < array.length; i ++){
4.           if (array[i] == v)  return i;
5.        return -1;
6.     }

7.     public TreeNode build (char[] preorder, char[] inorder) {
8.        if (preorder.length == 0)  return null;

9.        //step 1: find the root key and create a root node
10.       char rootValue = preorder[0];
11.       TreeNode root = new TreeNode(rootValue);

12.       //step 2: find the index of root key in in-order
13.       int leftSize = find(inorder,rootValue);

14.       //step 3: build left and right sub-tree's
15.       char[] leftPreorder = Arrays.copyOfRange(preorder,1,1+leftSize);
16.       char[] leftInorder = Arrays.copyOfRange(inorder,0,leftSize);
17.       root.left = build (leftPreorder, leftInorder);

18.       char[] rightPreorder = Arrays.copyOfRange(preorder,1+leftSize,preorder.length);
19.       char[] rightInorder = Arrays.copyOfRange(inorder,leftSize+1,preorder.length);
20.       root.right = build(rightPreorder, rightInorder);

21.       return root;
22.    }
23.}
```

```
1.  class TreeNode {
2.        int val;
3.        TreeNode left, right;

4.        TreeNode(int x) {
5.           val = x;
6.        }
7.  }
```

```
1.  public class Test{
2.     public static void main(String[] args) {
3.        char[] preOrder =
4.                 new char[]{'A','B','C','D','E'};
5.        char[] inOrder =
6.                 new char[]{'C','D','B','A','E'};
7.
8.        BTreeBuilder b = new BTreeBuilder();
9.        TreeNode root = b.build(preOrder, inOrder);
10.       System.out.println("build successfully");
11.    }
12.}
```

What's the time complexity?

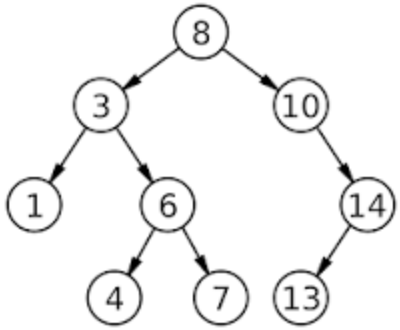
Binary Search Tree (BST)

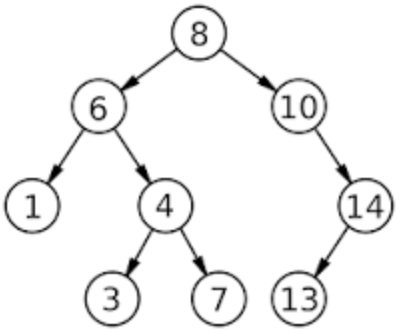
Outline

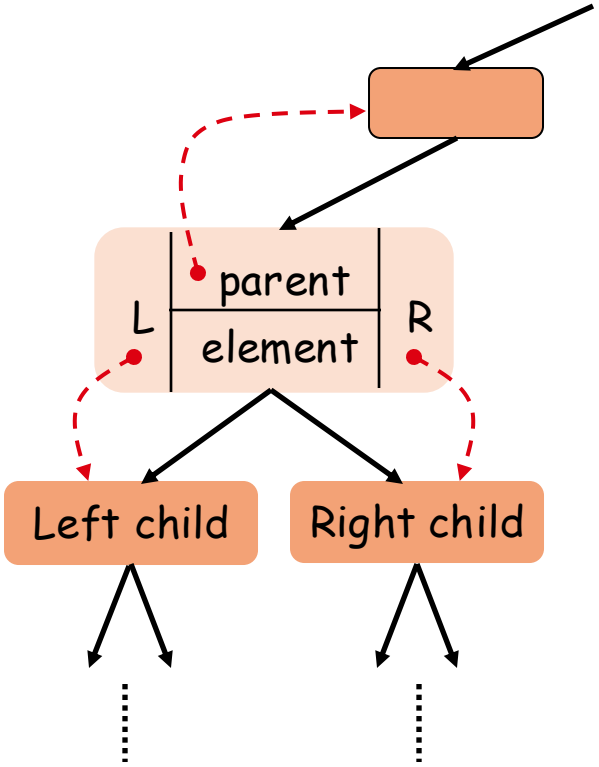
- Binary search tree (BST)
- Operations on BST
  - Search a key, find the minimum/maximum, find successor/predecessor
  - Insert, delete

Binary search tree (BST) property

- BST is a tree such that for each node T,
  - the key values in its left subtree are smaller than the key value of T
  - the key values in its right subtree are larger *than* the key value of T







Binary Search Trees (BSTs)

- Support many dynamic set operations
  - find, findMin, findMax, predecessor, successor, insert, delete
- Running time of basic operations on binary search trees
  - On average: $\Theta(\log n)$
    - The expected height of the tree is $\log n$
  - In the worst case: $\Theta(n)$
    - The tree is a linear chain of n nodes

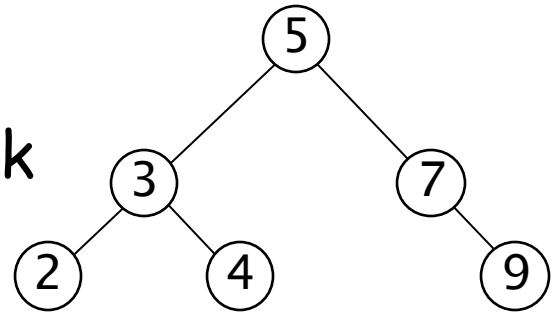
BST ADT

- Support many dynamic set operations
  - searchKey, findMin, findMax, predecessor, successor, insert, delete

- Running time of basic operations on BST
  - On average: $\Theta(\log n)$
    - The expected height of the tree is $\log n$
  - In the worst case: $\Theta(n)$
    - The tree is a linear chain of n nodes

Searching for a Key in BST

- Given a pointer to the root of a tree and a key k:
  - Return a pointer to a node with key k if one exists
  - Otherwise return NIL

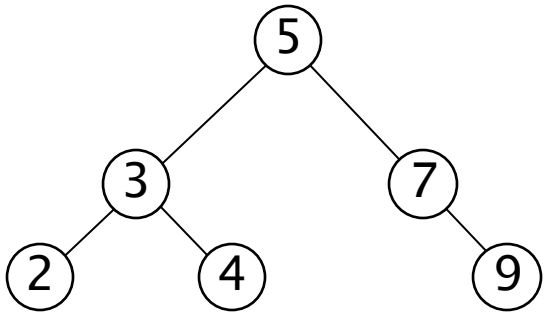


- Idea
  - Starting at the root: trace down a path by comparing k with the key of the current node:
    - If the keys are equal: we have found the key
    - If k < key[x] search in the left subtree of x
    - If k > key[x] search in the right subtree of x

Searching for a Key

find(x, k)

1. **if** x = NIL or k = key [x]
2. **then return** x
3. **if** k < key [x]
4. **then return** find(left [x], k)
5. **else return** find(right [x], k)



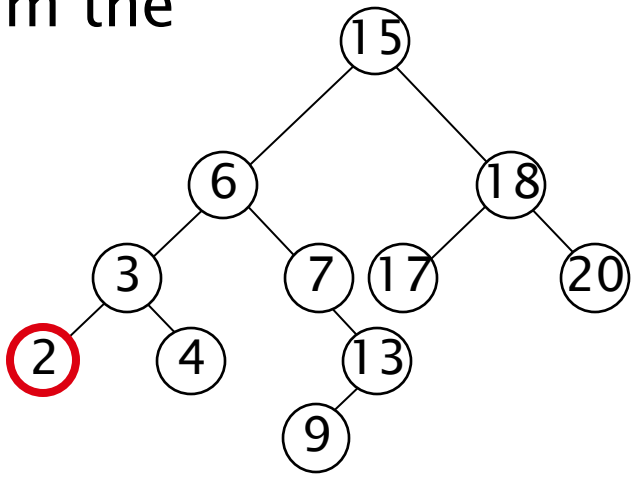
Running Time: *O* (h),
h – the height of the tree

Finding the Minimum

- Goal: find the minimum value in a BST
  - Following left child pointers from the root, until a NIL is encountered

findMin(x)

1. **while** *left* [x] ≠ NIL
2. **do** x ← *left* [x]
3. **return** x



Minimum = 2

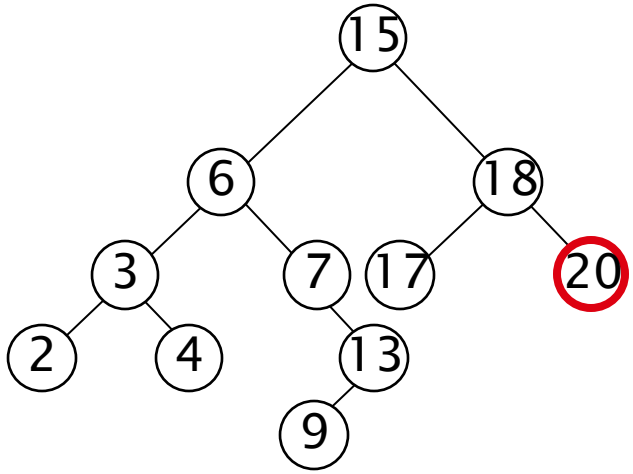
Running time: $O(h)$, h – height of tree

Finding the Maximum

- Goal: find the maximum value in a BST
  - Following right child pointers from the root, until a NIL is encountered

findMax*(x)*

1. **while** right [x] ≠ NIL
2. **do** x ← right [x]
3. **return** x



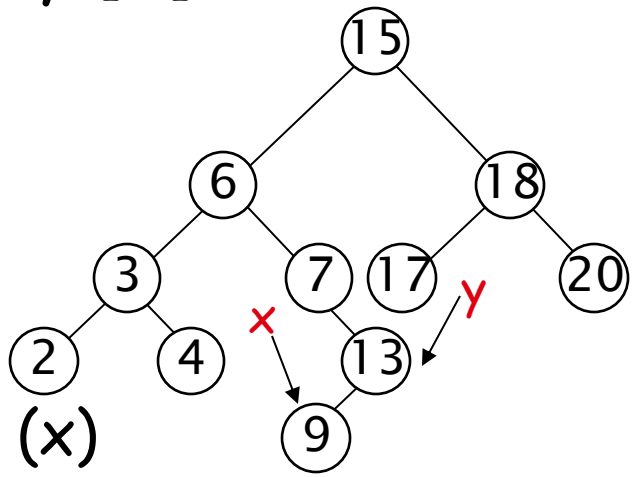
Maximum = 20

- Running time: *O*(h), h – height of tree

Successor

Def: $successor(x)$ = y, such that key [y] is the smallest key $>$ key [x]

- *E.g.:* $successor(15) = 17$
  $successor(13) = 15$
  $successor(9) = 13$

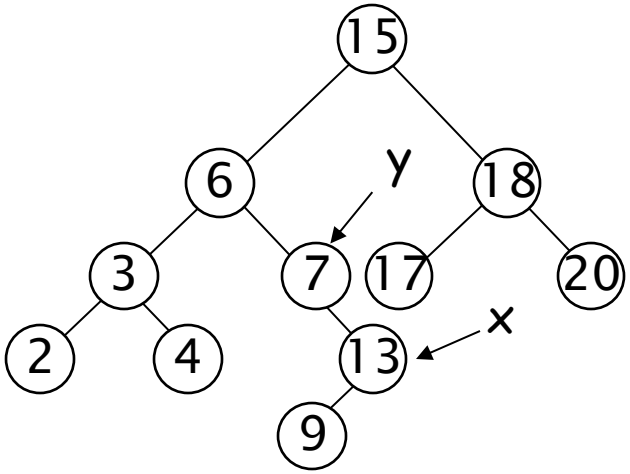


- Case 1: right (x) is non empty
  - $successor(x)$ = the minimum in right (x)
- Case 2: right (x) is empty
  - go up the tree until the current node is a left child: $successor(x)$ is the parent of the current node
  - if you cannot go further (and you reached the root): x is the largest element

Successor

successor*(x)*

1. **if** right [x] ≠ NIL
2. **then return** findMin(right [x])
3. y ← p[x]
4. **while** y ≠ NIL and x = right [y]
5. **do** x ← y
6. y ← p[y]
7. **return** y

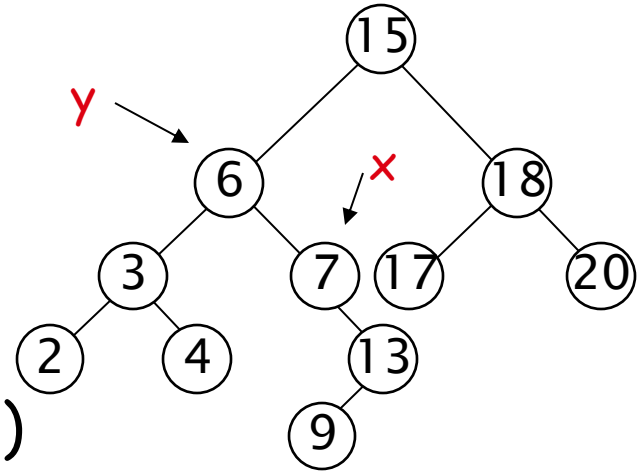


Running time: *O* (h), h – height of the tree

Predecessor

Def: *predecessor* $(x) = y$, such that key [y] is the biggest key < key [x]

- *E.g.:* *predecessor* $(15) = 13$
  predecessor $(9) = 7$
  predecessor $(7) = 6$



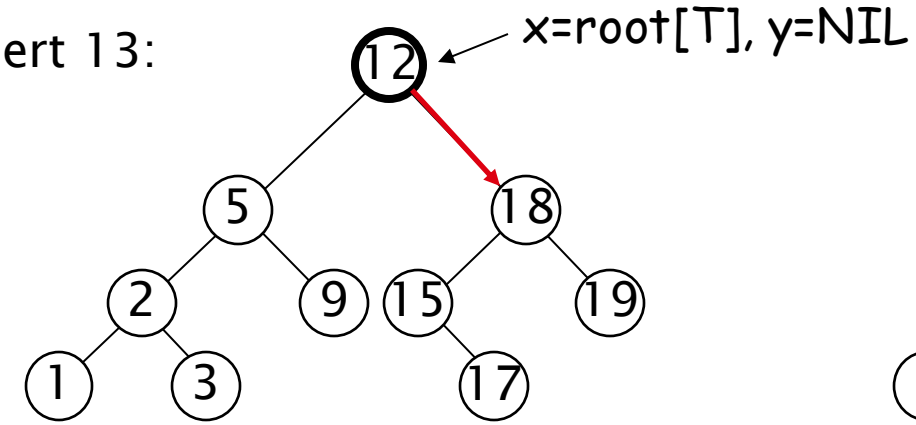
- Case 1: left (x) is non empty
  - *predecessor* (x) = the maximum in left (x)
- Case 2: left (x) is empty
  - go up the tree until the current node is a right child: *predecessor* (x) is the parent of the current node
  - if you cannot go further (and you reached the root): x is the smallest element

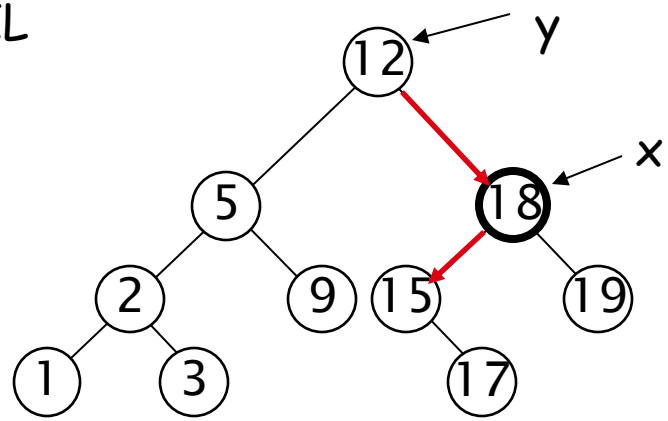
Insert

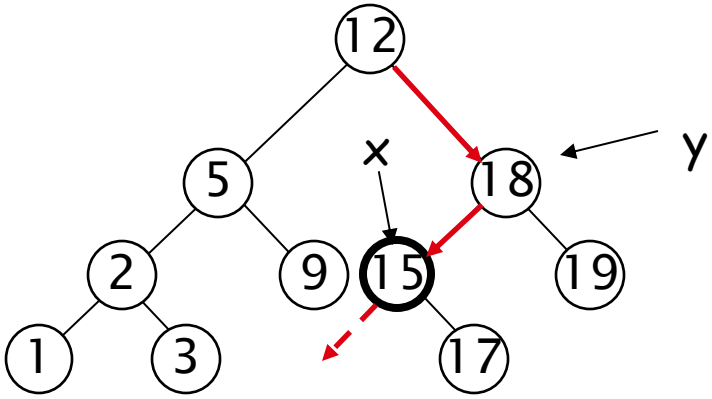
- Goal:
  - Insert value v into a binary search tree
- Find the position and insert as a leaf:
  - If key [x] < v move to the right child of x,
    else move to the left child of x
  - When x is NIL, we found the correct position
  - If v < key [y] insert the new node as y's left child
    else insert it as y's right child
  - Beginning at the root, go down the tree and maintain:
    - Pointer x : traces the downward path (current node)
    - Pointer y : parent of x ("trailing pointer")

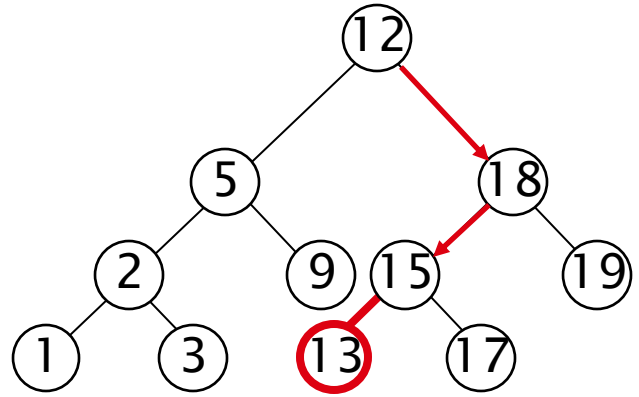
Example

Insert 13:



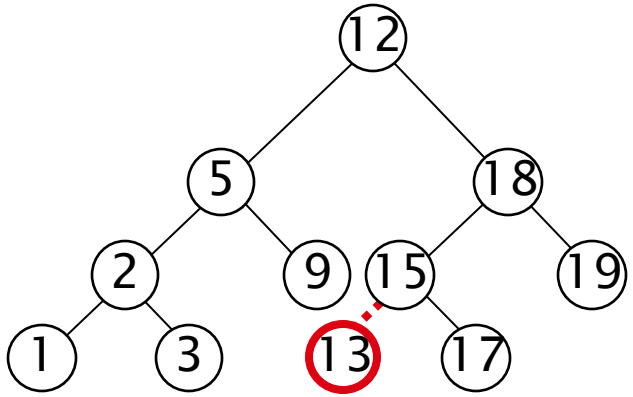






Insert algorithm

```
1.  y ← NIL
2.  x ← root [T]
3.  while x ≠ NIL
4.      do y ← x
5.          if key [z] < key [x]
6.              x ← left [x]
7.          else x ← right [x]
8.  p[z] ← y
9.  if y = NIL
10.     then root [T] ← z          ▷ Tree T was empty
11.     else
12.         if key [z] < key [y]
13.             left [y] ← z
14.         else
15.             right [y] ← z
```



Running time: $O(h)$

Exercise 1

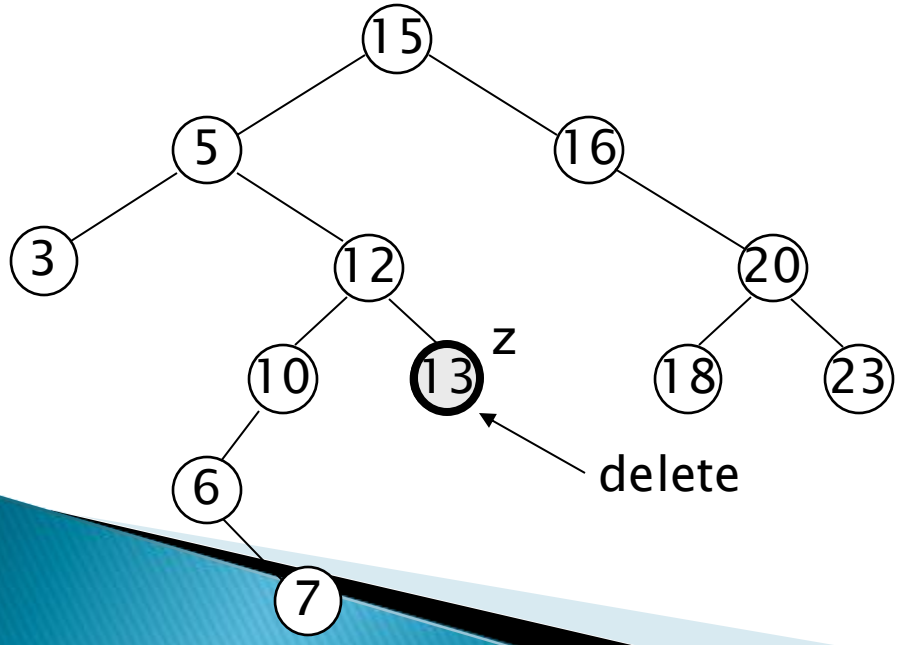
- Build a binary search tree for the following sequence

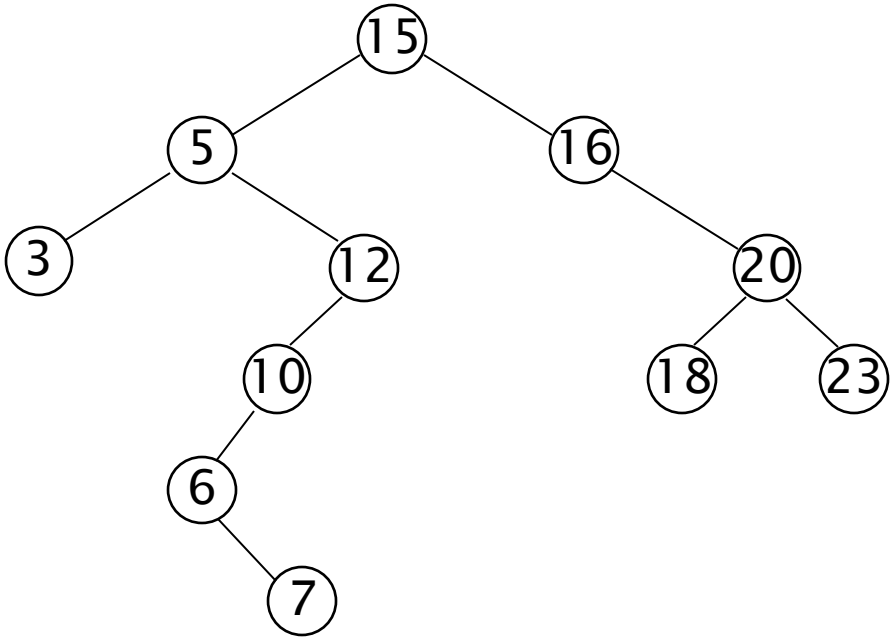
15, 6, 18, 3, 7, 17, 20, 2, 4, 13, 9

- Build a binary search tree for the reverse of this sequence

Deletion

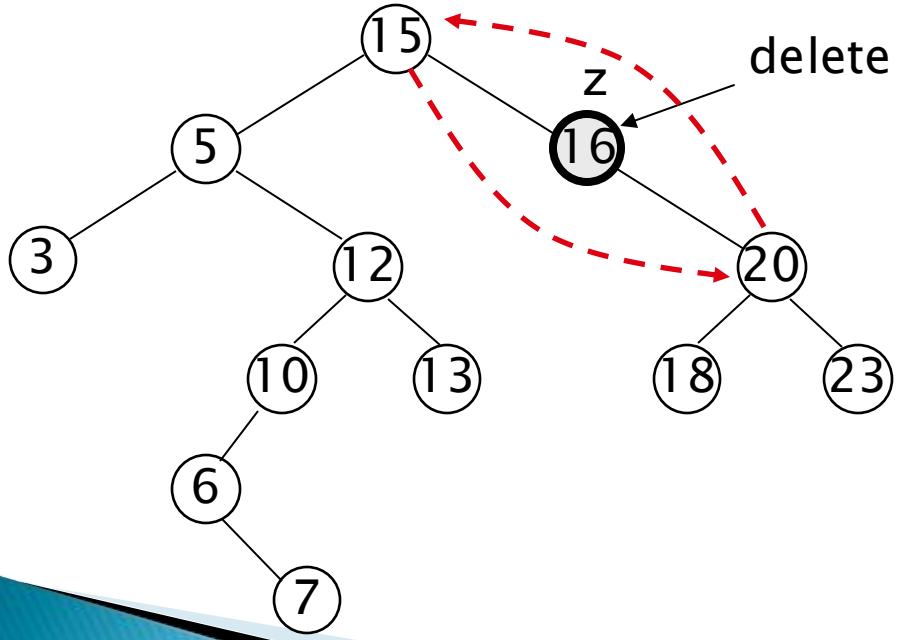
- Goal:
  - Delete a given node **z** from a binary search tree
- Idea:
  - **Case 1: z** has no children
    - Delete **z** by making the parent of **z** point to NIL

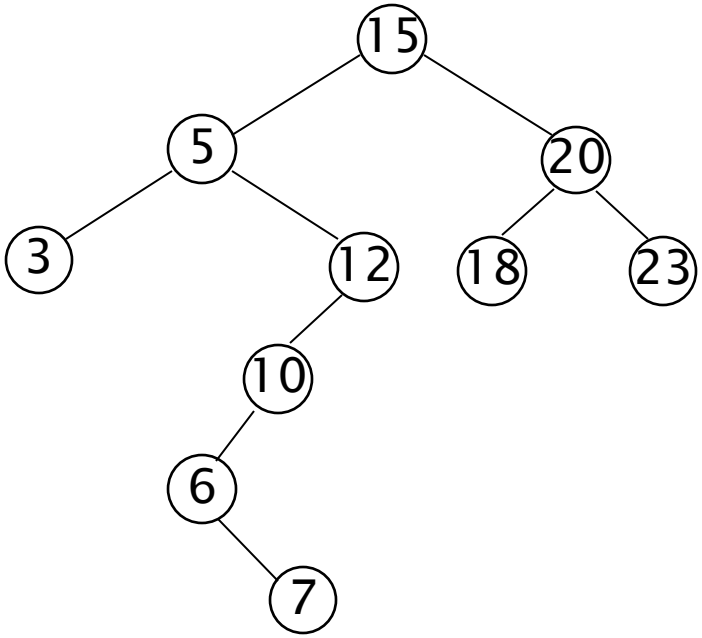




Deletion

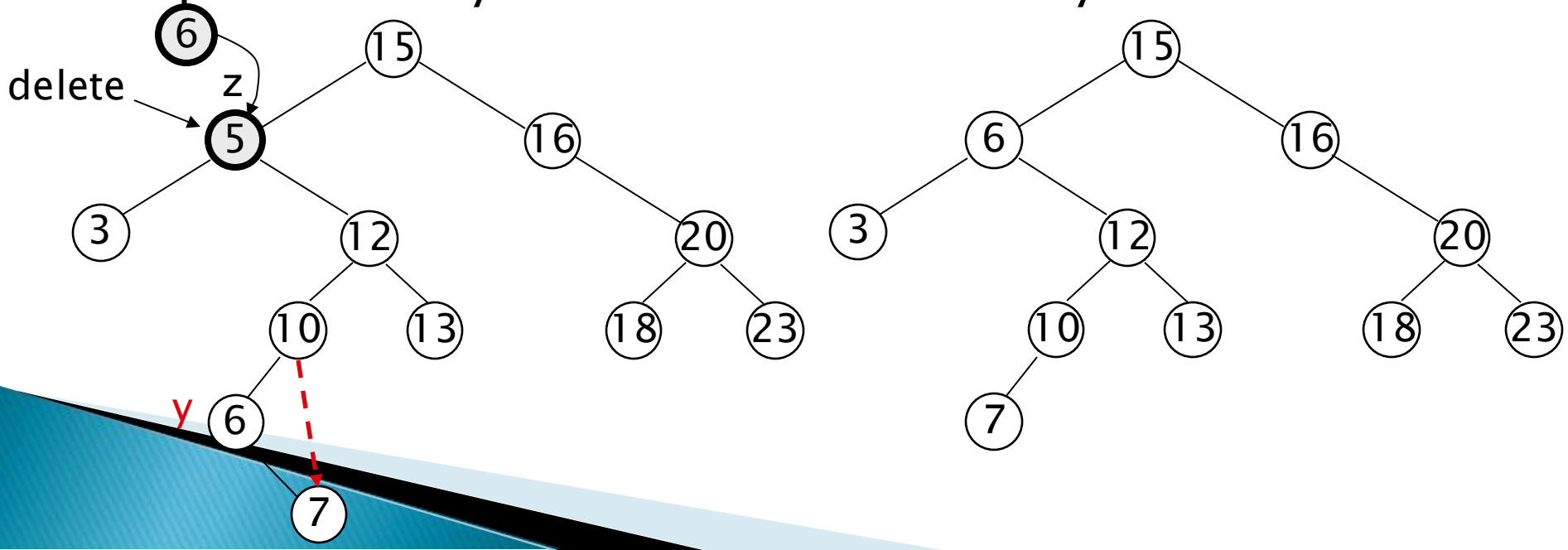
- **Case 2: z has one child**
  - Delete z by making the parent of z point to z's child, instead of to z and link the parent with the new child





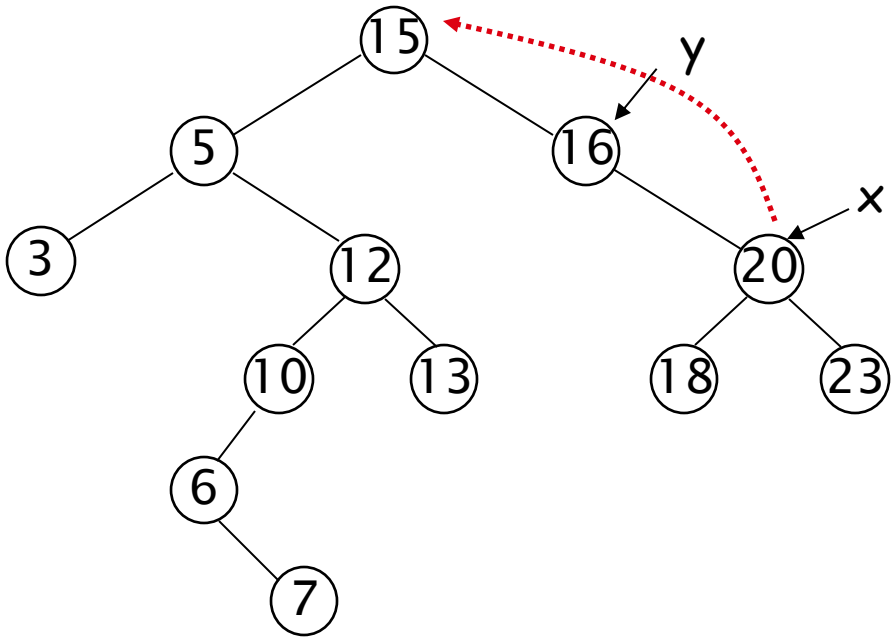
Deletion

- **Case 3: z has two children**
  - Find z's *successor y* (the leftmost node in z's right subtree)
  - y has either no children or one right child (but no left child), why?
  - Delete y from the tree (via Case 1 or 2)
  - Replace z's key and satellite data with y's



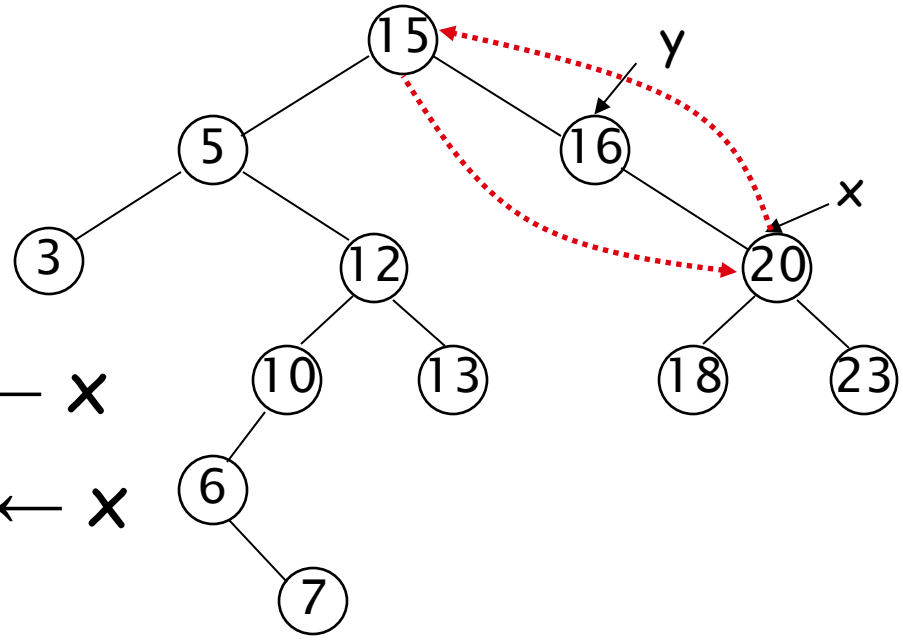
Deletion algorithm

1. **if** left[z] = NIL or right[z] = NIL
2. **then** y ← z — z has one child
3. **else** y ← TREE-SUCCESSOR(z) — z has 2 children
4. **if** left[y] ≠ NIL
5. **then** x ←left[y]
6. **else** x ← right[y]
7. **if** x ≠ NIL
8. **then** p[x] ← p[y]



Deletion algorithm cont.

```
9.  if p[y] = NIL
10.    then root[T] ← x
11.    else if y = left[p[y]]
12.              then left[p[y]] ← x
13.              else right[p[y]] ← x
14. if y ≠ z
15.    then key[z] ← key[y]
16.             copy y's satellite data into z
17. return y
```



Running time: $O(h)$

Binary Search Trees – Summary

- Operations on binary search trees:
  - SEARCH $O(h)$
  - PREDECESSOR $O(h)$
  - SUCCESOR $O(h)$
  - MINIMUM $O(h)$
  - MAXIMUM $O(h)$
  - INSERT $O(h)$
  - DELETE $O(h)$
- These operations are fast if the height of the tree is small – otherwise their performance is similar to that of a linked list

Binary search trees vs linear lists

Operation	BST	Sorted-array-based List	Linked List
Constructor	O(1)	O(1)	O(1)
IsFull	O(1)	O(1)	O(1)
IsEmpty	O(1)	O(1)	O(1)
RetrieveItem	O(logN)*	O(logN)	O(N)
InsertItem	O(logN)*	O(N)	O(N)
DeleteItem	O(logN)*	O(N)	O(N)

*assuming h=O(logN)

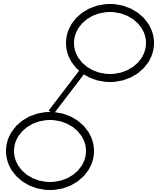
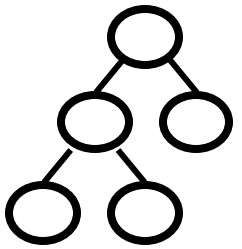
The issues in BST

- After a series of DELETION, the above algorithm favors making the left sub-trees deeper than the right

- One solution:
  - Try to eliminate the problem by randomly choosing between the smallest element in the right sub-tree and the largest in the left when replacing the deleted element (not rigorous and not prove it yet!!)

Exercise 1: count leaves

Example:

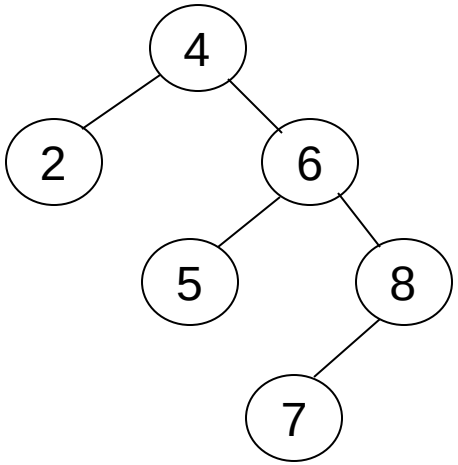
A NULL binary tree has 0 leaf node	 A tree with 1 node has 1 leaf node	 No. of leaf nodes = 1	 No. of leaf nodes = 3

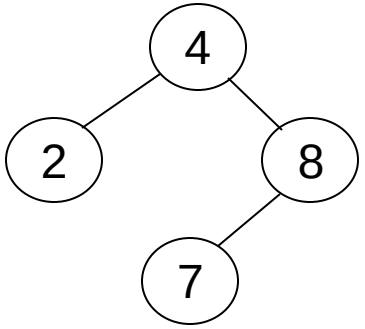
```
//To count the number of leaf nodes

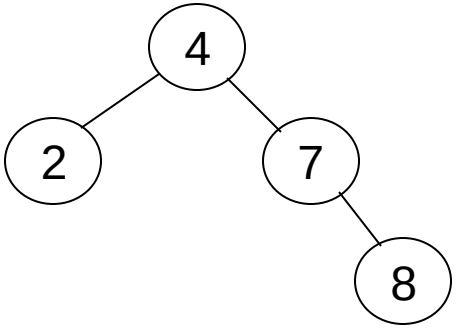
int Mytree::count_leaf(TreeNode* p)
{
      if (p == NULL)
            return  0;
      else if ((p->left == NULL) && (p->right == NULL))
            return  1;
      else
            return  count_leaf(p->left) + count_leaf(p->right);
}
```

Exercise 2

- In a binary search tree, are the insert and delete operations commutative?
  - insert(a) then insert(b) ⇔ insert(b) then insert(a)?
  - delete(a) then delete(b) ⇔ delete(b) then delete(a)?







Delete 5 and then 6 or delete 6 and then 5